SOFTOMATIC
TECHNOLOGY SERVICES

TRAINER GUIDE · GENAI + PROJECTS · THU 29 OCT

GenAI Session 6
Teach Docker and Compose

> Every student leaves with their API and a Postgres + pgvector database running from one command, and can explain image vs container, service names, volumes and secrets.

SESSION 3
12:45 to 2:15
Group A + Group T

SESSION 4 (REPEAT)
2:15 to 3:45
Group B + Group T

STUDENT NOTES
GenAI06 PDF
share before class

THE CLASS IN FIVE MOVES

01 Recap and the hook
02 Recipe, kit, meal
03 Live demo: Compose + /ready
04 Students build on their laptops
05 **Break it, exit ticket, tonight**

THE 90-MINUTE PLAN

Run sheet for today

Teach the same 90 minutes twice: 12:45 for Group A + T, 2:15 for Group B + T. Group T sits in both; in the repeat they finish their own setup in the first 40 minutes, then help a Group B student who is stuck.

TIME	WHAT HAPPENS	YOUR GOAL
0–10	Recap quiz on Session 5 + the hook ("it works on my machine")	Hands up: whose Docker Desktop is running?
10–25	Board: recipe → kit → meal; read the Dockerfile together; layer cache	Students can say build vs run
25–40	Board: Compose network, the localhost trap, volumes, secrets in three places	Students can say why db:5432
40–55	Live demo: docker-compose.yml, up --build, /health, /ready, pgvector check	Everyone has seen it work once
55–80	Students build: compose file, /ready, test; you walk the room	Compose running on every laptop that can run Docker
80–90	Break it on purpose (stop db), exit ticket, tonight's task	Every student has seen /ready go 503

Six things every student must leave with

Build vs run
You build an image; you run a container.

Dockerfile order
Libraries before code, so rebuilds are fast.

Service names
db:5432 inside Compose; localhost means "me".

Volumes
down keeps data; down -v deletes it.

/health vs /ready
Alive vs can serve; 503 when db is down.

Secrets
.env locally; never in Git or the image.

SCOPE FOR TODAY

Today is the Docker half of the "Deploy on day one" session doc: Compose, /ready, secrets. **Do not start CI or deployment**; that is GenAI 7 tomorrow (Fri 30 Oct), which ends with the week-2 tag. The template already has a Dockerfile, so students read it, they do not write it.

THE EVENING BEFORE

Prepare so the Wi-Fi does not decide the class

The two images are large downloads. Eighty laptops pulling them at 12:45 on classroom Wi-Fi is the most likely way this session fails.

Checklist

- [ ] Wednesday: post in the group: "Install Docker Desktop and open it once before Thursday. Windows: run `wsl --update` first." Link the student notes, section 11.
- [ ] On your laptop, with good internet: `docker pull pgvector/pgvector:pg16` and `docker pull python:3.12-slim`
- [ ] Save them to a USB drive for offline copying: `docker save pgvector/pgvector:pg16 python:3.12-slim -o g6-images.tar`. Students load with `docker load -i g6-images.tar`
- [ ] Your own copy of the starter repo with docker-compose.yml, /ready and the test from the student notes; run `docker compose up --build` once at home
- [ ] Check that hub.docker.com and pypi.org open on the classroom Wi-Fi; keep a phone hotspot ready
- [ ] The GenAI06 student notes PDF shared in the group before 12:45

At 12:45 and 2:15: a 30-second poll

"Hands up if Docker Desktop is running right now." Note the others. Send them to the no-Docker options (student notes, section 11) at minute 40 instead of debugging installs during your teaching time.

No-Docker options to name

1. GitHub Codespaces (about 60 free hours a month on a 2-core machine). **2.** Pair with a neighbour and run on their laptop. **3.** uvicorn without a database: /ready says "not configured".

DO NOT

Do not spend class time installing Docker on one laptop. An install with WSL updates and a restart can take 20 minutes. Note the name, send the student to an option above, and help after class.

RECAP, HOOK AND TWO PICTURES

Open, then teach build vs run

Cold-call names for the recap. Then tell the story, draw the recipe picture and read the Dockerfile together on the projector.

ASK THE CLASS

Q. What does a retry with backoff do when the LLM call times out?
Expect: Tries again after waiting longer each time, a limited number of times.

Q. When does your code switch to the fallback model?
Expect: When the first model fails after its retries (error or timeout).

Q. Name two things you log for every LLM call.
Expect: Model, tokens in/out, time taken, cost.

SAY THIS

"Your friend clones your repo and spends 20 minutes installing Python and Postgres. The CI server and the host have the same problem."
"Docker ships the whole box: code, Python, libraries, start command. It runs the same on every machine."
"Two words: you BUILD an image, you RUN a container. Recipe, kit, meal."

DRAW ON THE BOARD

Dockerfile (recipe) → docker build → **Image** (sealed kit, read-only) → docker run → **Container 1, 2, 3** (meals, throw-away).
Then, bottom to top: FROM python · COPY requirements + pip install · COPY app · USER + CMD. **"Code changes daily, so it goes last."**

ASK THE CLASS

Q. You change one line of app/main.py and rebuild. Does pip reinstall everything?
Expect: No; requirements.txt did not change, so that layer is cached.

Q. Why does the CMD use ${PORT:-8000}?
Expect: Hosts tell the app which port to use through $PORT; locally it falls back to 8000.

COMPOSE AND SECRETS

Teach names, volumes and keys

Draw the network picture, then ask the localhost question before you give the answer. Most students get it wrong; that is the point.

DRAW ON THE BOARD

A dashed box labelled **docker compose up**. Inside: **api** → **db**, arrow labelled **db:5432**. Under db: **volume pgdata**. Outside: **laptop** → **localhost:8000** → **api**.
Then a red arrow from api looping back to itself: **"localhost = me"**.

ASK THE CLASS

Q. Inside the api container, what does localhost:5432 reach?
Expect: The api container itself; nothing listens there, so "connection refused". Use db:5432.

Q. docker compose down, then up. Is the data still there?
Expect: Yes, in the pgdata volume. Only down -v deletes it.

Q. Why depends_on with condition: service_healthy?
Expect: So the api waits until pg_isready says Postgres is ready.

SAY THIS

"Settings that change between places come from environment variables. On your laptop: .env, which is git-ignored and docker-ignored."
"Never ENV LLM_API_KEY in a Dockerfile, never COPY .env into the image. Anyone who can pull the image can read it."
"If a key ever reaches Git, deleting the file is not enough. Rotate the key at the provider."

LINK TO SYSTEM DESIGN

Students met health checks and load balancers in SD L5. Say: "/ready is what the load balancer asks before it sends traffic to this server."

LIVE DEMO, 15 MINUTES

Demo script, step by step

Project your terminal and editor side by side. Type slowly and say each line aloud. Students watch; they build next.

1. **Show the Dockerfile and .dockerignore; point at .env in .dockerignore**
   Say: "The key never enters the image."

2. **Create docker-compose.yml (paste from the student notes, section 07); read the db block, then the api block**
   Say: "db by name, a volume for the data, a health check."

3. **Show .env has LLM_PROVIDER=fake**
   ```
   cat .env
   ```
   Say: "Fake model today, so nothing costs money and nothing needs a key."

4. **Start everything**
   ```
   docker compose up --build
   ```
   Say: "Watch: db becomes healthy first, then the api starts."

5. **Second terminal: liveness**
   ```
   curl localhost:8000/health
   ```

6. **Add /ready (student notes, section 08), rebuild, call it**
   ```
   docker compose up --build
   curl localhost:8000/ready
   ```
   Say: "database: ok. SELECT 1 worked."

7. **Check pgvector**
   ```
   docker compose exec db psql -U app -d rag -c "CREATE EXTENSION IF NOT EXISTS vector; SELECT extversion
   FROM pg_extension WHERE extname = 'vector';"
   ```
   Say: "A version number means Week 3 search will work."

8. **Show status and logs**
   ```
   docker compose ps
   docker compose logs api | tail -20
   ```
   Say: "When something breaks, read the last 20 lines first."

9. **Run the tests outside Docker**
   ```
   pytest -q
   ```
   Say: "The test has no database, so /ready says not configured. That is correct."

IF THE DEMO PULL IS SLOW

Do not wait in silence. You pulled the images last night; if your laptop still pulls, switch to the hotspot and keep talking through the compose file.

WALK THE ROOM: 25 MINUTES

What to check on each laptop

Go row by row. Check the list in order and fix the first problem you see; do not debug one laptop for ten minutes.

Seven checks, in order

- [ ] docker-compose.yml in the repo root, next to the Dockerfile
- [ ] DATABASE_URL uses **db:5432**, not localhost
- [ ] .env exists and is not in `git status`
- [ ] `docker compose ps` shows db as **healthy**
- [ ] /ready returns `database: ok`
- [ ] psycopg[binary] in requirements.txt; database_url in Settings
- [ ] test_ready_without_database added; `pytest -q` green

Red flags

- [ ] Secrets typed into docker-compose.yml or the Dockerfile
- [ ] Student ran `down -v` "to fix it"
- [ ] Editing code but never using --build, then "my change is not showing"
- [ ] Copied the compose file with tabs instead of spaces (YAML error)

Use Group T

In the 2:15 repeat, Group T students who already have Compose running sit with one Group B student each. Most problems are a missing .env, a busy port or a typo in YAML.

Students without Docker

They still write the compose file, /ready and the test, run `pytest`, and push. Then they run it in Codespaces or on a neighbour's laptop. They write which option they used in PROGRESS.md.

DIFFERENT PROJECTS, SAME WORK

All 84 projects use the same template, so today's files are identical for every engine (REG, ENT, FIN, CODE, CIT). Only the app name in .env differs.

EXPECT THESE, HAVE THE ANSWER READY

Troubleshooting in class

PROBLEM	WHAT TO TELL THE STUDENT
"port is already allocated" for 8000	An old uvicorn or container. Ctrl+C it, or `docker ps` then `docker stop NAME` ; or map "8001:8000".
"port is already allocated" for 5432	A local Postgres is running. Stop it, or map "5433:5432".
/ready gives 503 just after start	Postgres still starting. Wait for "healthy" in `docker compose ps` ; check `condition: service_healthy` is there.
/ready gives 503 "connection refused" to localhost	DATABASE_URL points to localhost. Change to db:5432.
"env file .env not found"	`cp .env.example .env` , set LLM_PROVIDER=fake, run again.
ModuleNotFoundError: psycopg (in the container)	Add `psycopg[binary]` to requirements.txt; `docker compose up --build` .
"WSL 2 installation is incomplete"	`wsl --update` in an administrator terminal, restart, reopen Docker Desktop.
"Virtualization must be enabled"	Needs Intel VT-x / AMD-V (SVM) on in BIOS. Not a class-time fix: send to Codespaces.
"Cannot connect to the Docker daemon"	Docker Desktop is not running. Open it and wait for "Engine running".
yaml: line N: found character that cannot start any token	Tabs in YAML. Use spaces; copy again from the notes.
Image pull stuck	Use the USB tar: `docker load -i g6-images.tar` , or a hotspot.
"no space left on device"	`docker system df` , then `docker image prune` ; free a few GB on the laptop.

RULE FOR SECRETS

If you see a real API key in a student's compose file, Dockerfile or a commit, stop: move it to .env, and if it was pushed, the student rotates the key at the provider today.

CLOSE THE LOOP

Break it, exit ticket and cold-calls

Everyone with Compose running does the experiment at the same time. Then the exit ticket, aloud.

1 **Everyone: stop the database and call both endpoints**

```
docker compose stop db
curl -i localhost:8000/health   # 200
curl -i localhost:8000/ready    # 503
```

Say: "Alive, but not ready. The load balancer stops sending traffic here; it does not restart it."

2 **Start it again**

```
docker compose start db
```

Say: "A few seconds later /ready is 200 again. Nothing restarted the api."

ASK THE CLASS

Q. Exit 1. What do you build, and what do you run?
Expect: Build an image, run a container.

Q. Exit 2. Which command deletes your database data?
Expect: docker compose down -v

Q. Exit 3. Where does the API key live on your laptop, and where must it never be?
Expect: In .env; never in Git, the Dockerfile or the image.

Cold-call: interview questions (all PATTERN-ONLY)

QUESTION	STRONG ANSWER HAS
How do you manage secrets for your application?	.env git-ignored locally · secrets in CI and host settings · never in the image · rotate on leak
Why Docker Compose, and how do containers find each other?	One command · one network · DNS by service name (db:5432) · volume for state · health-gated startup
Image vs container? (added for this session)	Image = read-only built package · container = running instance · many containers per image

HOMEWORK AND HOW TO CHECK IT

Tonight: Dockerfile + docker-compose.yml working

Read this to the class before they leave. The nightly GitHub check at 9:45 PM looks for today's commits.

SAY THIS BEFORE THEY LEAVE

"Tonight: push docker-compose.yml, the /ready route and the new test. Add a Run with Docker section to your README with three lines: cp .env.example .env, docker compose up --build, curl localhost:8000/ready. pytest must pass and .env must not be committed. No Docker? Write which option you used in PROGRESS.md."

Marking: 10 points per student

WHAT YOU CHECK ON GITHUB	POINTS	QUICK WAY
docker-compose.yml with db (pgvector:pg16, volume, healthcheck) and api (build, env_file, depends_on)	3	Open the file
DATABASE_URL uses db:5432	1	Search "db:5432"
/ready route in app/main.py, returns 503 on failure	2	Open main.py
test_ready_without_database in tests/	1	Open tests/test_health.py
"Run with Docker" section in README	1	README
No .env or real key anywhere in the repo	2	File list; search the repo for "API_KEY="

10 = all present. A real key in the repo is 0 for that row and the student rotates the key today, whatever the rest scores.

TOMORROW (GENAI 7, FRI 30 OCT)

CI builds this same Dockerfile and smoke-tests /health; students deploy the image to a free host and push the week-2 tag. A student whose compose file works today has an easy Friday.

KEEP THIS OPEN WHILE YOU TEACH

One-page trainer cheat card

Timings

S3 12:45–2:15 · Group A + T

S4 2:15–3:45 · Group B + T (repeat)

Nightly check 9:45 PM

The two pictures

Recipe → kit → meal (Dockerfile → image → container)

Compose box: api → db:5432, volume pgdata

localhost = "me"

Demo commands

`docker compose up --build`

`curl localhost:8000/health` · `/ready`

`docker compose ps` · `logs api`

Experiment

`docker compose stop db`

/health 200 · /ready 503

`docker compose start db` → 200

Secrets lines

.env: git-ignored and docker-ignored

Never ENV key / COPY .env in Dockerfile

Leaked → rotate at the provider

Top fixes

Port busy → stop it or map 8001/5433

localhost in DATABASE_URL → db

No .env → cp .env.example .env

No Docker

Codespaces (~60 h/month free, 2-core)

Neighbour's laptop · or uvicorn, no db

Never install during teaching time

Not today

CI, pre-commit, deploy, live URL

week-2 tag

All in GenAI 7, Fri 30 Oct

TRAINER CHECKLIST

Four habits for this class

Pre-pull the images
USB tar or hotspot; never let Wi-Fi decide the class.

Ask before you tell
The localhost question first; the wrong answers teach.

Do not install during class
Send no-Docker students to an option, help after.

Break it on purpose
Every student sees /health 200 while /ready is 503.

> By 3:45 every student can start their API and database with **one command**, and explain why it works the same on every machine.

THIS SESSION

WEDNESDAY	12:45 · 2:15	AFTER 9:45 PM
Ask students to install Docker	Poll, teach, demo, walk	Check compose files on GitHub